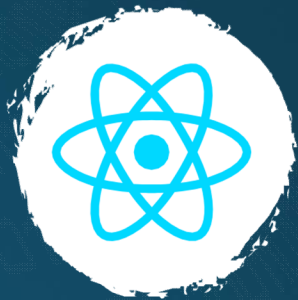


Frontend Interview Questions/Answers



Shahzaib Akram
(full stack Developer)

[Linkedin.com/in/shahzaibakram](https://www.linkedin.com/in/shahzaibakram)



Questions



1. What is the purpose of the **DOCTYPE** declaration in HTML?
2. Explain the difference between **<div>** and **** in HTML?
3. How do you link an external CSS stylesheet to an HTML document?
4. What is the significance of the alt attribute in the **** tag?
5. Describe the difference between **GET** and **POST** methods in HTML forms.
6. What is the purpose of the **<meta>** tag, and how is it used?
7. How can you create a hyperlink that opens in a new tab or window?
8. What is the semantic difference between **** and ****, and **** and **<i>**?
9. Explain the concept of HTML5 semantic elements and provide examples.
10. How does the **<iframe>** tag work, and what is it commonly used for in HTML?

Answer



1. `<!DOCTYPE>` – Defines HTML version and helps browsers render the page correctly.
2. `<div>` vs `<span>` – `<div>` is block-level, `<span>` is inline-level.
3. Link CSS – Use `<link rel="stylesheet" href="style.css">` in the `<head>`.
4. alt attribute – Provides alternative text for images when they can't load.
5. GET vs POST – GET sends data via URL, POST sends data securely in the body.
6. `<meta>` tag – Supplies metadata like charset, viewport, or description to the browser.
7. Open link in new tab – Add `target="_blank"` to the `<a>` tag.
8. `<strong>/<em>` vs `<b>/<i>` – First pair gives semantic meaning, second is styling only.
9. Semantic elements – Tags like `<header>`, `<article>`, `<section>` give content structure.
10. `<iframe>` tag – Embeds another webpage into the current page, often for videos/maps.

Questions



1. What is the **box model in CSS** and how does it work?
2. Explain the difference between **margin** and **padding** in CSS.
3. How can you center an element horizontally and vertically in CSS?
4. What is the purpose of the z-index property in CSS?
5. Describe the difference between **inline** and **block-level elements** in CSS.
6. What is the difference between **display: none** and **visibility: hidden** in CSS?
7. How do media queries work in responsive web design with CSS?
8. Explain the CSS selector specificity and how it affects styling.
9. What is the **clearfix hack**, and why is it used in CSS?
10. How can you include **external fonts** in a web page using CSS?

Answer



1.Box Model – Element has content, padding, border, margin layers.

2.Margin vs PaQdding – Margin is outside, padding is inside the element.

3.Centering – Use Flexbox: justify-content and align-items.

4.z-index – Sets stacking order of elements.

5.Inline vs Block – Inline stays in-line, block takes full width.

6.display: none vs visibility: hidden – none removes, hidden hides but keeps space.

7.Media Queries – Apply styles for different screens: @media.

8.Specificity – Resolves conflicting styles based on selector priority.

9.Clearfix – Fixes float issues by clearing floated children.

10.External Fonts – Use @font-face or Google Fonts link.

Question



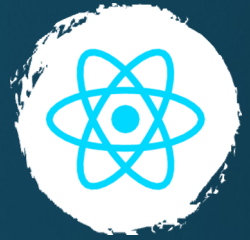
1. What is the difference between **let**, **const**, and **var** in JavaScript?
2. Explain the concept of **closures** in JavaScript.
3. How does **prototypal inheritance** work in JavaScript?
4. What is the purpose of the **this** keyword in JavaScript?
5. How do you handle **asynchronous** operations in JavaScript?
6. What is **event delegation** in JavaScript?
7. Explain the difference between **==** and **===** in JavaScript.
8. How can you create a new object in JavaScript?
9. Describe the concept of **hoisting** in JavaScript.
10. What is the use of the **map** and **filter** functions in JavaScript?

Answer



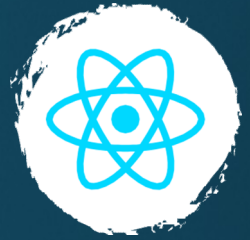
1. **let, const, var** – **let** and **const** are **block-scoped**, **Const** is **immutable**, **var** is **function-scoped**.
2. **Closures** – A function **remembers** variables from its **outer scope**, even after the outer function ends.
3. **Prototypal Inheritance** – Objects **inherit** properties from other objects via the **prototype chain**.
4. **this keyword** – Refers to the **context object** the function is called on.
5. **Asynchronous operations** – Handled using **callbacks**, **Promises**, or **async/await**.
6. **Event Delegation** – Use a **single event listener** on a parent to handle events on child elements.
7. **== vs ===** – **==** does **type coercion**, **===** checks for **strict equality** (type + value).
8. **Creating objects** – Use **object literals**, **new Object()**, or **constructor functions**.
9. **Hoisting** – Variables and functions are **moved to the top** of their scope before execution.
10. **map and filter** – **map** **transforms each item**, **filter** **selects items that match a condition**.

Questions



1. What is **JSX** in React and why is it used?
2. Explain the difference between **state** and **props** in React.
3. How does React handle **component lifecycle** methods?
4. What is the **significance of keys** in React lists?
5. What is the purpose of the **virtual DOM** in React?
6. How do you handle forms in React?
7. Explain the concept of **lifting state up** in React.
8. What are **React hooks** and how do they differ from class components?
9. Describe the role of the **useEffect** hook in React.
10. How does **routing** work in React applications?

Answer



1. **JSX** – **JS** + **HTML syntax** in **React** for **UI building**.
2. **State** vs **Props** – **State** is **internal**, **props** are **external** & **read-only**.
3. **Lifecycle Methods** – **React** uses **hooks** (e.g., **useEffect**) to manage **component phases**.
4. **Keys in Lists** – Help **React** **track and update** list items efficiently.
5. **Virtual DOM** – A **lightweight copy** of the **DOM** for **efficient updates**.
6. **Forms** – Use **controlled components** with **state** handlers.
7. **Lifting State Up** – Move **shared state** to the **closest common parent**.
8. **Hooks** vs **Classes** – **Hooks** use **functions** + **state**, **classes** use **lifecycle methods**.
9. **useEffect** – Handles **side effects** like **fetching**, **subscriptions**, etc.
10. **Routing** – Use **React Router** to handle **page navigation** via **components**.

Any more Information
Contact us



Shahzaib Akram
(full stack Developer)
[Linkedin.com/in/shahzaibakram](https://www.linkedin.com/in/shahzaibakram)